Appendix H — Concepts study guide — the hard concepts taught properly

Source: `GITLAB_STUDY_GUIDE.md`

Study Guide — the concepts, taught properly, not just scripted

This is different from the other docs. Those tell you *what to say*. This one is for actually understanding the five hardest concepts so you could derive the answer under follow-up pressure, not just recite it. Each section: the idea, why it exists, the mechanics, a worked example, a self-test, and — where it matters — something to actually build.

Read one section a day, in order. Don't skip the self-tests; if you can't answer one cold, re-read the mechanics, don't move on.

---

1. The permission gate: grounding vs. permission

The idea, slowly

Before an agent's proposed action is allowed to run, something has to check it. Most people describe this check as one fuzzy thing: "make sure the request is valid." That's wrong, and it's wrong in a way that matters. There are **two completely independent questions**, and a plan can pass one while failing the other:

- **Grounding** asks: *is this real?* Does the user/record/entity the plan refers to actually exist? Is the request well-formed against the system's actual current state?
- **Permission** asks: *is this caller allowed?* Regardless of whether the target is real, does *this specific caller* have the right to perform *this specific operation* on *this specific resource*?

Why they have to be separate — the part that's easy to get wrong

Picture two different bad requests:

1. The model plans to grant a **real, existing user admin access**. The caller asking for this has no authority to grant admin to anyone. Grounding passes (the user is real) — but permission must catch it, because the caller isn't allowed.
2. The model plans to grant access to a **user called "Jsmith" who doesn't exist** — a hallucinated name. The caller asking happens to be someone who legitimately could grant access to a real user. Permission would pass — but grounding must catch it, because the target isn't real.

One check, written as "validate the request," cannot catch both of these. You need both checks, run independently, in sequence. That's the entire argument, and it's the sentence that makes an interviewer sit up: "*grounding asks if it's real, permission asks if the caller's allowed — one check can't do both.*"

The full mechanics — four stages, not two

In practice there are two more stages after grounding and permission:

```
model proposes an action
        |
        ▼
```

```
1. GROUNDING   — do the referenced entities exist? is the request coherent against live state?
        │  fail → reject (hallucinated / malformed)
        ▼
2. PERMISSION  — is this caller allowed this operation on this resource?
        │  fail → reject (unauthorized)
        ▼
3. APPROVAL    — is this high-impact enough to need a human? if so, pause and wait.
        ▼
4. CONTRACT    — is the call shaped exactly as the tool's schema requires?
        │  fail → reject (malformed call, even from an authorized caller)
        ▼
   action executes
```

Each stage catches a *different* failure. Grounding catches hallucination. Permission catches unauthorized-but-real requests. Approval adds a human checkpoint for consequence, independent of authorization. Contract catches a well-authorized caller whose call is still shaped wrong.

Worked example — trace it yourself

A Salesforce admin agent is asked: "*Freeze the user john.doe@example.com.*"

1. **Grounding**: does a user with that email exist in this org right now? (A live lookup, not a cached assumption.)
2. **Permission**: does the caller who issued this request have "Manage Users" rights in this org? (Checked against the caller's actual role/policy — not inferred from the conversation.)
3. **Approval**: freezing a user is a state-changing, moderately consequential action — does policy require a human to confirm before it runs, or is this pre-approved for this caller?
4. **Contract**: is the freeze call shaped exactly as the freeze-user tool's schema expects — right field names, right types — regardless of steps 1–3 having passed?

Self-test — answer out loud, no notes

1. Give a concrete example (not the one above) where grounding passes but permission fails.
2. Give a concrete example where permission passes but grounding fails.
3. Someone answers "the agent validates the request before executing" — what's missing from that answer, in one sentence?
4. Which of the four stages would catch a syntactically perfect call made by someone with full rights, targeting a record that doesn't exist?

---

2. The action-selection funnel

The idea, slowly

An agent with 5 tools can just show the model all 5 and ask it to pick. An agent with 300 tools can't — showing the model all 300 every time is slow, expensive, and the model's accuracy drops as the list grows (it's a harder discrimination problem, and more tokens to reason over). The funnel is how you keep tool selection fast and accurate as the catalog grows without bound.

The mechanics

Two stages, doing two different jobs:

- **Stage A — narrow, cheap, parallel.** For each step the plan needs, run a fast vector search against the *entire* tool catalog. This isn't the model doing the thinking — it's retrieval, the same mechanism as document RAG, just applied to tools instead of text. Output: a short list, typically 3–5 candidates per step, not 300.
- **Stage B — reason, expensive, once.** A single LLM call now sees only the pre-filtered candidates for every step at once, plus the full context (the original goal, any requirements gathered, the candidate lists) and picks the best tool per step, in one grounded pass.

Why this is the right shape, not just *a* shape

If you skipped Stage A and just dumped the whole catalog into the prompt every time: token cost scales with catalog size, latency scales with catalog size, and accuracy *degrades* with catalog size because the model has a harder discrimination problem. If you skipped Stage B and just picked the top vector-search hit: you'd lose the model's ability to reason about *which* of the 5 close candidates is actually right for this specific step's phrasing and context — vector similarity alone isn't precise enough for action selection. You need retrieval to narrow, and reasoning to pick. Neither stage alone is sufficient.

Worked example

Tool catalog: 300 actions. Task: "retrieve the ID of the user to clone." Stage A vector-searches that one step's description against all 300 actions, returns something like `[IdentifyMetadataRecordByName, QueryRecords, ExecuteEntityRead]` — three candidates, all plausible by text similarity. Stage B's single compile call, which already has the full task context (this is step 1 of a 3-step clone operation), picks `IdentifyMetadataRecordByName` because it's the one that resolves a name to an ID directly, which is what this specific step needs — a distinction vector similarity alone couldn't make confidently.

Self-test

1. Why can't you skip Stage A and just show the model everything?
2. Why can't you skip Stage B and just take vector search's top hit?
3. As the tool catalog grows from 50 to 5,000 actions, which stage's cost grows, and which stays roughly flat? Why?
4. What's one way this funnel could fail silently (Stage A's candidate list misses the actually- correct tool)? What would you check to catch that?

---

3. Session persistence & crash recovery — you've actually built this

Why this section is different

The earlier docs called this a "studied, not shipped" gap. That's wrong for ResolveFlow specifically — you built real durable persistence there: LangGraph `interrupt()` for pausing on human approval, with state checkpointed to Postgres via `AsyncPostgresSaver`, confirmed against the actual repo. The honest gap is narrower than it was framed: you haven't shipped this *at Salesforce*, but you have shipped it, for real, in a system you can show. Reframe your answer accordingly — don't undersell work you actually did.

The idea, slowly

A session is one in-progress multi-step task with its own history. Making it *durable* means it can survive a crash or a restart and pick up exactly where it left off — not start over.

The mechanics — snapshot, not replay

Two possible designs: **replay** the full event log from the start every time you need to resume, or **snapshot** the state periodically and resume directly from the latest snapshot. Replay is tempting because it's "simpler" conceptually, but it has a fatal flaw for anything with side effects: replaying a tool-call event could **re-trigger the real side effect** — re-sending an email, re-granting access, re-posting a comment. A snapshot restores state directly, without re-executing anything. This is the same reason databases do crash recovery via checkpoints plus a redo log, not by replaying every transaction from table creation.

Your version of this, precisely

In ResolveFlow: `interrupt()` pauses the graph at the approval node; the state up to that point is checkpointed to Postgres. On resume, the checkpoint is loaded directly — the graph does not re-run the diagnosis or retrieval nodes that already completed, it resumes exactly at the approval boundary with their outputs already in state. That's the snapshot pattern, live.

Self-test — this time, answer from your own code

1. Where exactly in your graph does the checkpoint get written, and what triggers it?
2. If the process crashes *between* writing the checkpoint and the human approving, what happens on restart? Walk through it.
3. Why would replaying the whole event log instead of loading the checkpoint be actively unsafe for this specific graph — name the real side effect that could double-fire.
4. What's NOT durable in your design — is there any in-memory state that would be lost on a crash that the checkpoint doesn't capture?

---

4. Compaction — the real gap, and the project to close it

The idea, slowly

Every model has a fixed context window. Most agent tasks finish well inside it. But a long-running or always-on agent — a CI bot, an agent that keeps working across many approval cycles — can accumulate more history than the window holds. Compaction is how a session outlives its window: once the conversation gets large enough, summarize the older turns into something short, and keep only the recent turns verbatim.

The trade-off — this is the part that actually matters

Compaction trades **detail for endurance**. A specific fact from 200 turns back might survive compaction only as a vague line in a summary, or vanish entirely. So the engineering question isn't "does it summarize" — it's **"what do you keep verbatim vs. compress, and how do you make sure a detail that turns out to matter later isn't permanently lost?"** The real answer: critical facts get written to an external store (a retrieval index, a structured field in state) *before* they're at risk of being compacted away, so they can be pulled back even after the live conversation history no longer contains them. That's the same shape as the core RAG trade-off you already know — deciding what to retrieve vs. what to leave out of context — just applied to an agent's own history instead of a document corpus.

The project — build a real, scoped version of this in ResolveFlow this week

This turns "studied, not built" into "built a working reference implementation," which is a materially stronger thing to say in the room. **I'm not going to write this for you** — same as always with ResolveFlow — but here's the scope, broken into steps you drive:

Goal: when a ResolveFlow thread's accumulated state crosses a size threshold, compact the older history into a short summary, keep the most recent N turns verbatim, and make sure nothing decision-critical (the original issue, the current plan, any approval state) is lost.

Design questions to work through yourself before writing code (this is the actual hard part — the code is the easy part once these are answered):

1. What counts as "state" in your graph right now that could grow unbounded — the full message history? Tool call results? Both? Look at your `GraphState` TypedDict.
2. What's the threshold that triggers compaction — a message count, a token estimate, something else? What's cheap enough to check on every node transition without adding real latency?
3. What must **never** be compacted away, because it's load-bearing for correctness, not just context? (Hint: think about what `execute` and `independent_review` actually read from state — if a compacted summary loses one of those fields, what breaks?)
4. Where does the "don't lose it" data go — a dedicated field in state that's excluded from compaction, or written out to Postgres/your retrieval store separately?
5. How do you test this — what's the smallest reproducible scenario that proves compaction happened, old detail is gone from the live context, but the thing you decided must survive actually survived?

Suggested shape for a first pass (still your call, not a prescription): a `compact` node that runs conditionally before the node that would otherwise see an oversized state, summarizes everything except the last N turns and a small set of protected fields, and writes the summary back into state in place of what it replaced.

When you're ready to start, come back and we'll work through it the normal way — you drive the design and the code, I review and push on the weak spots, same as the rest of ResolveFlow.

Self-test (concept-level, answerable before the project exists)

1. State the trade-off in one sentence, the way you'd say it in the room.
2. Name one concrete fact in a ResolveFlow thread that would be catastrophic to lose to compaction, and why.
3. Why is "write critical facts to external memory before compacting" a better mitigation than "just compact less aggressively"?

---

5. Data Cloud retrieval — the grounding pipeline

The idea, slowly

An agent that reasons about an org's own configuration needs to look that configuration up — "grounding" the model in live, tenant-specific state rather than letting it guess. This is a retrieval pipeline like any other, with one added wrinkle: it has to be **tenant-isolated**, because the exact same question asked by two different orgs must retrieve two different answers, and under no circumstances leak one org's metadata into another's.

The mechanics — five hops, in order

1. **Identity/token exchange** — before any retrieval call, the caller's org identity gets exchanged for a scoped token specific to that tenant. This is what makes isolation possible at all; without a scoped token, there's no way to guarantee the retrieval layer only sees this org's data.
2. **Gateway routing** — the scoped call is routed through a central gateway into the retrieval platform.
3. **Retrieval, split by content type** — general product/help content goes through one index; operational/procedural documents (the "how do I do X" documents) go through a separate index built specifically for that purpose. They're kept separate because they answer different kinds of questions and have different freshness requirements.
4. **Result return** — retrieved chunks flow back through the same gateway to the planner.

5. **Ingestion, separately and continuously** — none of this works if the index is empty or stale, so there's a separate, periodic pipeline that extracts source documents and refreshes the index, running independently of any live request.

Why tenant isolation has to happen *before* retrieval, not after

If you retrieved broadly and then filtered out other tenants' results afterward, you'd depend on that filter never having a bug — and a single bug would be a cross-tenant data leak, not just an inefficiency. Scoping the token *before* the retrieval call means the retrieval layer physically cannot see another tenant's data in the first place — the isolation is structural, not a filter you hope never fails.

Self-test

1. Why does tenant isolation need to happen before retrieval, not as a post-filter?
2. What happens to a request if the ingestion pipeline is behind — does retrieval fail, or return stale results silently? Why does that distinction matter?
3. Why are general product documentation and operational/procedural documents kept in separate indexes instead of one combined one?

How to use this before Oct 9

- Read one section a day, Oct 3–7 (this maps onto what `GITLAB_WEEK_PLAN.md` already schedules for those days — use this doc as the actual study material for those blocks, not just the bullet points).
- Do every self-test out loud, without looking back at the mechanics. If you hesitate, that's the signal to re-read, not to move on.
- Start the compaction project (§4) no later than Oct 5, so you have real code — even a rough first pass — to describe by Oct 8's mock interview, not just a design.
